

What Happens When You Type a URL in Your Browser?

A Beginner-Friendly Tutorial on DNS, Ports, and Web Requests

When we open a browser and type a website address like:

`www.google.com`

and press **Enter**, many networking processes happen behind the scenes within milliseconds.

To a user, it looks simple:

Type URL → Website opens

But internally, the browser performs several networking operations including:

- DNS Resolution
- TCP Connection Establishment
- TLS/SSL Security Handshake
- HTTP Request Transmission
- Server Response Processing
- Webpage Rendering

This tutorial explains the complete process step by step.

1. Understanding the Problem

Computers do not understand domain names such as:

`www.google.com`

They communicate using **IP addresses**, such as:

`142.250.183.36`

So when a user enters a URL, the browser must first convert the domain name into an IP address.

This conversion process is called:

Domain Name System (DNS) Resolution

DNS acts like the internet's phonebook.

2. Step-by-Step Journey of a URL Request

Let us assume the user enters:

`www.google.com`

Step 1: Browser Checks Its Cache

The browser first checks whether it has already resolved this domain recently.

It searches its internal DNS cache.

Example:

`www.google.com → 142.250.183.36`

If found, the browser uses this IP address immediately.

If not found, it moves to the operating system cache.

Step 2: Operating System Checks DNS Cache

The operating system (Windows/Linux/macOS) stores recently resolved domain names.

The browser asks the OS:

Do you know the IP address of `www.google.com`?

If the answer is yes, the IP is returned.

If not, the system sends a request to a DNS resolver.

Step 3: Query Sent to Recursive DNS Resolver

The computer sends a DNS query to a **recursive resolver**.

This resolver is usually provided by:

- Internet Service Provider (ISP)
- Google DNS (8.8.8.8)
- Cloudflare DNS (1.1.1.1)

Protocol and Port Used

DNS typically uses:

Protocol Port Number

UDP 53

TCP 53

Why UDP?

UDP is faster.

Why TCP?

Used when:

- Response is too large
- Zone transfers occur
- Reliability is required

The computer asks:

What is the IP address of `www.google.com`?

Step 4: Recursive Resolver Checks Its Cache

The recursive resolver checks whether it already knows the answer.

If cached:

It immediately replies.

If not, it starts querying DNS hierarchy.

3. DNS Hierarchy Explained

DNS works in layers.

The hierarchy is:

Root DNS

↓

TLD DNS

↓

Authoritative DNS

Step 5: Root DNS Server

The recursive resolver first contacts a **Root DNS Server**.

It asks:

Where can I find information about `www.google.com`?

The root server does not know the exact IP address.

It responds:

Ask the `.com` TLD server.

Role of Root DNS Server

The root server directs requests to the correct Top-Level Domain (TLD) server.

Examples of root servers:

- `A.root-servers.net`
- `B.root-servers.net`

There are 13 logical root server clusters globally.

Simple Analogy

Root DNS is like asking:

Which country is this address in?

Step 6: TLD DNS Server

The resolver now contacts the **`.com` TLD server**.

It asks:

Where can I find `google.com`?

The TLD server responds:

Ask Google's authoritative nameserver.

Example:

`ns1.google.com`

`ns2.google.com`

Role of TLD Server

TLD stands for:

Top-Level Domain

Examples:

- `.com` , `.org` , `.edu` , `.pk`

Its job is to point toward the domain's authoritative nameserver. It does not know the final IP address.

Analogy

TLD is like asking:

Which city or area contains this address?

Step 7: Authoritative DNS Server

Now the resolver asks Google's authoritative DNS server:

What is the IP address of `www.google.com`?

The authoritative server replies:

`www.google.com` → `142.250.183.36`

Role of Authoritative DNS Server

This server contains the official DNS records.

It is the final source of truth.

4. Common DNS Records

A Record

Maps domain to IPv4 address

Example:

`www.google.com` → `142.250.183.36`

AAAA Record

Maps domain to IPv6 address

MX Record

Mail server record

Used for email routing

CNAME Record

Alias for another domain

Example:

`blog.site.com` → `hosting.provider.com`

Step 8: IP Address Returned to Browser

The recursive resolver sends the IP address back.

The OS caches it.

The browser caches it.

This improves speed for future requests.

Step 9: Browser Initiates Connection

Now the browser knows where to connect.

Example:

`142.250.183.36`

It opens a connection to the server.

Important Port Numbers

Service Port

DNS 53

HTTP 80

HTTPS 443

FTP 21

SSH 22

For websites:

HTTP

Port 80

HTTPS

Port 443

Modern websites mostly use HTTPS.

5. TCP Three-Way Handshake

Before data transfer, a TCP connection must be established.

This happens in three steps.

Step 1: SYN

Client sends:

Synchronize

Meaning:

Can we connect?

Step 2: SYN-ACK

Server replies:

Synchronize + Acknowledge

Meaning:

Yes, I am ready.

Step 3: ACK

Client sends:

Acknowledge

Meaning:

Connection established.

Visual Representation

Client → SYN → Server

Client ← SYN-ACK ← Server

Client → ACK → Server

This is called:

TCP Three-Way Handshake

6. TLS/SSL Handshake (HTTPS Security)

If the site uses HTTPS, encryption setup begins.

This is called:

TLS Handshake

During this process:

The browser and server exchange:

- Encryption algorithms
- Security certificates
- Session keys

Why is TLS Important?

It provides:

Confidentiality

Data cannot be read by attackers

Integrity

Data cannot be altered

Authentication

Ensures the server is genuine

7. Browser Sends HTTP Request

After connection is established, the browser sends:

```
GET / HTTP/1.1
Host: www.google.com
User-Agent: Chrome
```

Components

GET

Request method

/

Requesting homepage

Host

Target website

8. Server Processes Request

The server receives the request.

The web server software processes it.

Common web servers:

- Apache
- Nginx

The server may:

- Fetch database data
- Run backend code
- Access cache
- Generate webpage content

9. Server Sends Response

The server replies with an HTTP response.

Example:

```
HTTP/1.1 200 OK
Content-Type: text/html
Then sends webpage content.
```

Common HTTP Status Codes

Code Meaning

200	Success
301	Redirect
404	Not Found
500	Server Error

10. Browser Renders Webpage

The browser receives:

- HTML
- CSS
- JavaScript
- Images
- Fonts

It then renders the webpage.

This is what the user finally sees.

11. Additional Requests

After loading the main HTML page, the browser often sends additional requests for:

- Images
- Stylesheets
- JavaScript files
- APIs
- Fonts

Each may trigger more DNS lookups and TCP connections.

Complete Sequence Diagram

```
graph TD; A[User types URL] --> B[Browser Cache Check]; B --> C[OS Cache Check]; C --> D[Recursive DNS Resolver]; D --> E[Root DNS Server]; E --> F[TLD DNS Server]; F --> G[Authoritative DNS Server]; G --> H[IP Address Returned]; H --> I[TCP Three-Way Handshake]; I --> J[TLS Handshake]; J --> K[HTTP Request]; K --> L[Server Response]; L --> M[Browser Rendering];
```

Real-Life Analogy

Suppose you want to visit a friend.

Root DNS

Which country?

TLD DNS

Which city?

Authoritative DNS

Which exact house?

TCP Handshake

Knocking at the door

HTTP Request

Asking:

May I come in?

Server Response

Friend opens the door

Browser Rendering

You enter and see the room

Practical Commands for Students

Check DNS Resolution

`nslookup google.com`

Trace Route

Windows:

`tracert google.com`

Linux:

`traceroute google.com`

View DNS Cache

Windows:

`ipconfig /displaydns`

Key Takeaways

Students should remember:

DNS converts domain names into IP addresses

DNS hierarchy:

Root DNS

↓

TLD DNS

↓

Authoritative DNS

Websites use ports:

- 80 for HTTP
- 443 for HTTPS

TCP uses Three-Way Handshake

`SYN → SYN-ACK → ACK`

Browser sends HTTP request after connection setup

Server responds with webpage data

Final Summary

When a user types a URL and presses Enter:

1. Browser checks cache
2. OS checks cache
3. DNS resolution begins
4. Recursive resolver queries:
 - Root DNS
 - TLD DNS
 - Authoritative DNS
5. IP address returned
6. TCP connection established
7. TLS handshake secures connection
8. HTTP request sent
9. Server responds
10. Browser renders webpage

All of this typically happens in less than a second.